BANK NEGARA MALAYSIA COPA HACKATHON 2026 · MUST-WIN 10

Project SELARAS

Semantic **E**ngine for **L**inkage **A**nalysis across **R**egulatory **A**rtefacts & **S**tandards

AI that reads BNM's policy corpus the way a drafter does — and says what a draft aligns with, differs on, and is silent about. Every claim quotes the clause it rests on.

BNM MUST-WIN 10 · 2026 MW10

AI Roadmap

Stronger internal alignment on the use of AI to enhance delivery of the Bank's mandate.

1Q2026	5-year AI roadmap endorsed by the Board
1 / sector	Business process re-engineering initiative identified per sector, using AI
>15%*	Efficiency improvement across 10 supervisory processes from staff use of AI tools

SELARAS is the Open Finance workstream's contribution toward these MW10 targets.

Consistency is credibility

BNM's authority as a regulator rests on a body of policy that agrees with itself and with the world. Today that agreement is maintained by hand. SELARAS turns each of the four problems below into a system property.

1

BNM's credibility as a regulator rests on being **internally consistent** with its own body of policy and **externally aligned** with international standards.

Four artefact stages — discussion decks, discussion papers, exposure drafts, policy documents — must stay coherent with each other, with the Act, and with BIS, HKMA, MAS and ABS.

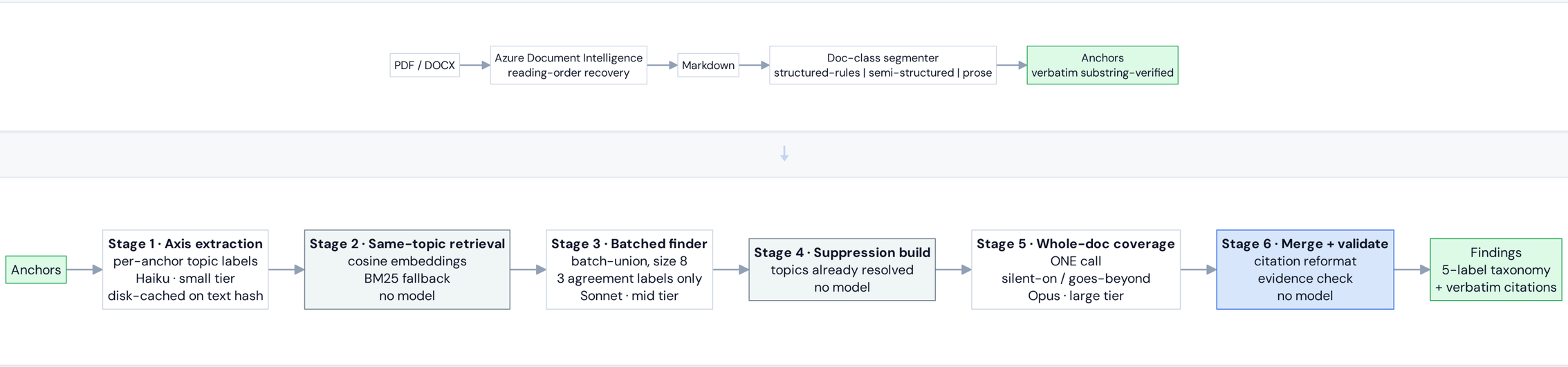
2 · PROBLEM STATEMENTS		3 · IMPACT
Manual assessment of overlapping areas of concern with other departments	→	Automatic detection of overlaps with other departments
Manual benchmarking of BCBS and other jurisdictions' policies	→	Optimised opportunity-gap analysis to explore and exploit established practices
Lengthy drafting lifecycle — drafters spend disproportionate time on routine tasks instead of higher-value work	→	Drafters' time reallocated to higher-value activities — review and scoping what needs to be assessed
Institutional awareness and domain expertise take years to develop, concentrated in individual drafters	→	Institutional awareness and domain expertise decoupled from individuals, embedded into the system

Comparing two documents, clause by clause

Problems 1 and 2 — manual overlap checks, manual benchmarking. Today a drafter reads BCBS, HKMA or another department's document end to end against their own draft. Checking every clause against every clause would take **160,000 AI calls**; reading both documents in one go is cheap but then nothing can be quoted precisely. Six steps get it to **9 AI calls per pair of documents**.

THE ENGINE, END TO END

9 AI calls per pair of documents



WHY THIS HOLDS UP

Searching alone can never find a gap

Search finds clauses that **exist**. The finding a drafter most needs is the one that doesn't. Nothing on our side to search for — so stage 5 reads both documents whole.

Cheap work runs on a cheap model

Tagging a clause isn't hard thinking — small model, and remembered. Judging a pair is — mid model. Only reading both at once uses the large model, **exactly once**.

There is no second AI checking the first

An earlier design had one model review another's work. We tested both and the simpler one matched it. **Half the calls, same answers** — measured, not assumed.

We fix typos, we never guess

A stray colon on a clause number is stripped and logged. A number that only half-matches a real clause is **marked unsupported** — we never guess which was meant.

Problems 1 & 2 — the reading a drafter does by hand today, done in 9 AI calls. Invent a clause number and the output is dropped, not published

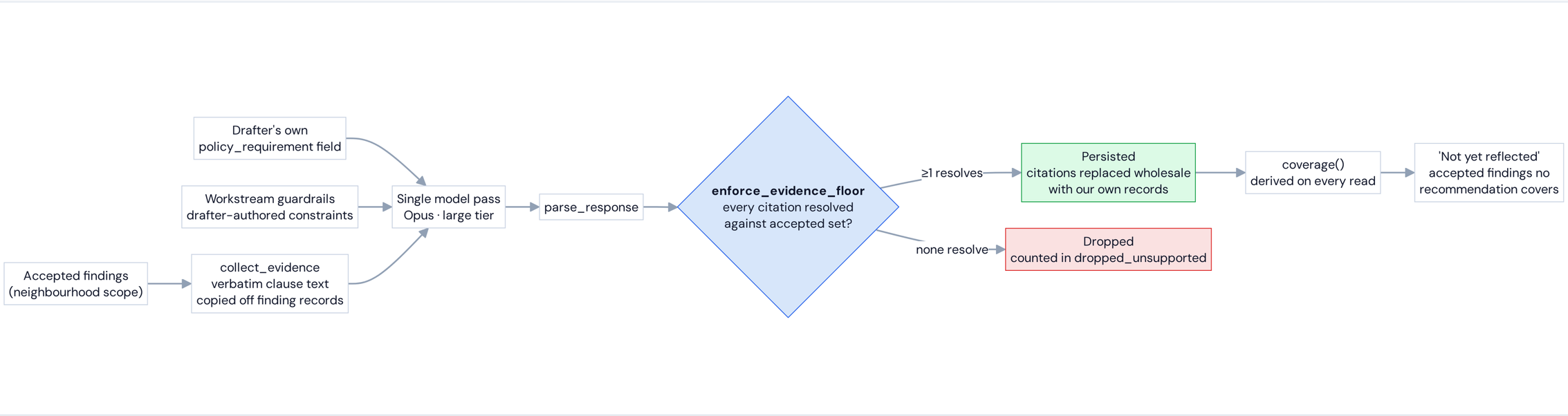
`aligns-with` `differs-on` `conflicts-with` `silent-on` `goes-beyond`

From findings to what the draft should say

Problem 3 — time lost to routine work. Finding linkages is half the job. The drafter is still left with thirty accepted comparisons — "our clause 8.4 is silent where HKMA's 4.2 is not" — and every word still to write. **Problem 4 — expertise stuck in one head.** This engine writes the first version, and writes the house rules down where the next drafter can read them.

HOW A RECOMMENDATION IS MADE — AND WHAT IT HAS TO SURVIVE

The check runs after the AI answers, not before



WHY THIS HOLDS UP

The quoting rule is built in, not asked for

We don't ask the AI to cite honestly and hope. Every quote is checked afterwards, and if none hold the recommendation is **deleted before it is ever saved**.

The headings are theirs, not ours

Recommendations are organised by the requirements the drafter wrote down. **Nothing written down → we stop, and say why.** They read back their own words.

The house rules leave one head and go on the record

Judgement built over years — how BNM cross-references, what scope a draft may claim — becomes something every future draft reads. **Only the drafter writes it.**

Drafting copilot

Two of the three layers are already built and tested. What is left is **joining the five commands to a backend that already exists** — composition, not new capability.

HOW THE COPILOT IS PUT TOGETHER

What you type one of five commands

The editor your live draft and whatever you have selected — **context only, never quotable**

The orchestrator — it knows which stage you are in

A command only runs after the one before it, and the drafter's playbook for that stage goes into every prompt.

Stage state machine

/explore-task	/brainstorm	/draft	/write	/deliver
Read what this document is, report what's missing from its profile.	Pull the accepted findings and recommendations, ask what to focus on.	Propose the section outline, map each finding to a section.	Write the prose for a section, quoting accepted evidence.	Completeness check, then send for maker–checker sign-off.
document profile playbook	accepted findings recommendations	recommendations house rules	clause index accepted findings	document profile playbook

The same gate, on the riskiest step

Every tool above is a route that already exists. **/write** is the one most likely to invent a clause — any quote it wasn't given is dropped and re-quoted from the real text.

`_validate_reply · built`

Prose lands in the draft at the cursor, or replacing the whole document — the drafter's choice

WHERE WE ACTUALLY ARE

• Built **The model backend**

Real model calls, live streaming, and the citation gate above — all written and tested. `engine/copilot.py`

• Built **The routes the app talks to**

The endpoints and the typed client the workspace calls are in place and covered by tests.

• The gap **Wiring the five commands up**

The five commands are scripted in the interface against real clauses. **What's left is joining them to the backend that already exists.**

WHY THE GAP IS SMALL

The drafter's playbook already reaches the prompt

The playbook screen writes their instructions and the backend already reads the right section per stage. **Only the command→stage dispatch is missing.**

Streaming is already solved

Prose streams word by word; quotes and draft snippets parse after it finishes. **Nothing to re-engineer.**

Feasible today, scalable by construction

Each claim below carries either the **evidence** that it already holds or the **feature** that makes it hold. Nothing here is aspirational.

FUNCTIONAL — Feasibility

- **UX is accurate to the user journey of policymaking**
  - FEATURE Node and edge analysis support the full lifecycle of policy artefacts:
  - Discussion decks → Discussion paper → Exposure draft → Policy document
- **Policy recommendations are practical**
  - EVIDENCE **67% of sampled recommendations are marked as valid by an FDI analyst.**
  - FEATURE Recommendations are classified based on input policy requirement.
- **Citation integrity is guaranteed**
  - FEATURE The finder pipeline is **forced** to return findings with citations; unsupported findings are **strictly dropped**.

FUNCTIONAL — Scalability

- **Extending to other departments needs no further configuration**
  - EVIDENCE Onboarding new departments only requires **document preparation and ingestion** rather than additional application logic.

0 lines of new application logic to onboard a department

8 node types · 3 edge types · 5 finding labels · One vocabulary, every department

NON-FUNCTIONAL — Feasibility

- **LLM cost is optimised**
  - EVIDENCE Three model tiers, each matched to the reasoning difficulty of its stage:

Haiku · small	Sonnet · medium	Opus · large
Concept retrieval — axis / topic extraction	Batched same-topic finder — `differs-on`, `conflicts-with`, `aligns-with`	Whole-document coverage finder — `silent-on`, `goes-beyond`

NON-FUNCTIONAL — Scalability

- **Cost scales with edges actually analysed, not with graph size**
  - FEATURE Linkage analysis is **dynamically evaluated on demand by the user**, so growth is **linear and bounded by demand**, preventing exponential computational overhead.

O(n) on edges the drafter opens not **O(n×m)** on the whole graph

Live demo workstream: **open-finance-pd-2026** — a real BNM exposure draft against RMiT, BIS, HKMA and the ABS/MAS API playbook

• Validated by an FDI policy analyst

Three risks, three answers

The failure modes that matter for a regulator are accuracy, confidentiality and accountability. Each is answered by a mechanism, not by a policy statement.

01 The AI gets it wrong

A finding cites a clause that doesn't exist, or misjudges one that does.

Citations are passed into a **deterministic validation guard**, not an AI model. The drafter can also **accept or reject any finding**.

Code-level validator | Human accept / reject on every finding | Unsupported findings dropped, never shown as fact

02 Confidentiality

Bank drafts or internal papers leave the Bank.

In scope — Prototype built using **public documents** for demonstration.

Out of scope — Hosting on the Bank's private infrastructure, deploying private AI models, and building an enterprise-grade application.

03 Accountability

A policy ships with an error and nobody owns the decision.

Phase 2 — **Maker-checker with a full audit trail.** Enforce segregation of duties where review actions need a checker, and **a checker cannot be the reviewer**.

Every transition logged with actor / timestamp / comment

The drafter decides. SELARAS **evidences** the decision — it never makes it.

Approving manager gives final sign-off

From one workstream to the institution's graph

Phase 1 is delivered and demonstrable today. Phase 2 does not add a new capability — it scales the one that already works.

Phase 1 — Prototype • Delivered

AI linkage-finding engine, live on the Open Finance policy workstream

Proves the model correctly cross-references a real BNM policy document (Exposure Draft) against external reference standards — **RMiT, BIS, HKMA, ABS/MAS API playbook**.

Five-label semantic classification with verbatim clause citation

`aligns-with` / `differs-on` / `conflicts-with` / `silent-on` / `goes-beyond`. Every finding quotes its source clause; **zero fabricated citations by design**.

Knowledge graph of the workstream — documents as nodes, structural edges

Replaces manual cross-referencing of policy documents with a navigable map.

AI drafting copilot

5-step workflow (`/explore` → `/brainstorm` → `/draft` → `/write` → `/deliver`) writes citation-grounded recommendations straight into the draft.

Demo runs on pre-filled data — live backend built (`engine/copilot.py`), UI orchestration wiring pending

Phase 2 — Next steps Institution-wide

Deduplicated institutional graph

Scales the one-workstream prototype into a **BNM-wide regulatory map**; merge all workstreams into one network where shared entities collapse to a single node.

Graph database + agentic Cypher retrieval

Natural-language questions traverse the whole graph in one query — e.g. which drafts are `silent-on` collateral types peer regulators (MAS, HKMA) already codified.

Jurisdiction / BIS clustering view

Group the graph by regulator and Basel standard; converts an internal linkage tool into a **benchmarking answer to "how does BNM compare globally."**

Maker-checker with a real per-finding audit trail

Enforced maker ≠ checker, every transition logged with **actor / timestamp / comment**.

M365 / SharePoint integration

Pull PDF / PPTX / XLSX directly from existing SharePoint sites, as an alternative to manual upload.

Phase 2 is **composition over existing capability** — the engine, the taxonomy and the guardrail are already built and tested

Cross-workstream linkage is the demo climax, not a preview

Built for one workstream. 80% applicable to three more BNM must-wins.

We built and demoed Open Finance. Nothing in the engine is specific to it — pointing it at another department is **uploading that department's documents**, not writing new code.

THE DEPARTMENTS AND THE EXACT DOCUMENTS THEY WOULD BRING

80% average applicability across these **three Blueprint 2026 must-wins** — same engine, no new application logic

MW2 Crisis resilience · MW5 Future-proof digital finance infrastructure · MW6 Financial sector strategy

FDI Built & demoed

Open Finance PD

FS

Banking System Financial Crisis Management Framework (Crisis Playbook), Conforming Loans Standards (CLS)

PFP

Operational Resilience Roadmap

PPD

DP on stablecoins and CBDC, CEO letter on tokenised deposit

JP4

Capital Adequacy Framework (Internal Ratings-Based Approach for Credit Risk) PD, Capital Adequacy Framework, Interest Rate Risk in the Banking Book PD

JDM

Economic Monetary Review (EMR), Monetary Policy Statement (MPS)

BNM **Financial sector blueprint (FSBP) for 2027-2030**

Only Open Finance is built today — the rest are **candidates**. The same **8 node types** already describe every document above

WHAT IT COULD BE USED FOR

1. **Benchmark BNM policy against BIS, MAS and HKMA**
   `international-standard`
2. **Catch clashes with another department before publishing**
   `internal-published`
3. **Check a draft against the law that governs it**
   `act-law`
4. **Stay consistent with past supervisory letters**
   `supervisory-letter`
5. **Keep OPR messaging consistent across statements**
   `internal-published`

Institution-wide map is a follow-on epic

Reviewed clause by clause, by the drafter who owns it

67% of sampled recommendations were assessed to be **valid by an FDI Policy Analyst**. All nine items, verbatim, below.

Validated verdict Analyst language emphasised in the source review — the recommendation or gap was accepted as valid.

Context / caveat Accepted with a Malaysia-context qualification, or explained as an **intentional** drafting choice.

Zero truncation Every recommendation, rationale, BNM action point and analyst comment is reproduced in full.

01 Add a public TSP registry and anti-impersonation requirement

The gap is relevant

RATIONALE

HKMA requires banks to publish all partnering TSPs and their products, and to monitor for fraudulent parties impersonating TSPs. ED requires TSP oversight but has no provision on publishing that list or on impersonation monitoring.

ACTION FOR BNM

Add a requirement for data providers to maintain and publish an up-to-date list of authorised TSPs accessing their systems, and an obligation to warn customers against impersonation fraud.

ANALYST COMMENTS

The gap is relevant, as a public registry of data consumers is useful, and Malaysia does not currently have one. However, it may be less relevant in Malaysia's context as participation is currently limited to FIs and does not yet include fintech-type participants.

Also, I checked the Hong Kong framework. Although HKMA uses the term "TSP" (third party service provider) and our ED uses the same term, they refer to different things. HKMA's TSP is closer to a data consumer in Malaysia, as it refers to an entity that accesses banking data to provide services. In contrast, Malaysia's TPSP refers to a third party engaged by an FI to support its operations (e.g. cloud, data analytics)

02 Consider extending data-sharing obligations to non-bank and big-tech players

The recommendation is good

RATIONALE

BIS identifies that banks must share customer data under open finance mandates while non-banks, fintechs, and big-techs holding financial-sector-relevant data face no equivalent obligation — creating market distortions. ED applies exclusively to defined FSPs.

ACTION FOR BNM

Assess whether a subsequent phase should extend reciprocal data-sharing obligations to non-regulated data holders, or explicitly state that the current asymmetry is an accepted design choice.

ANALYST COMMENTS

The recommendation is good, but ED currently only covers banking participants and does not include non-bank participants, the issue of unequal data-sharing obligations is not applicable

03 Maintain and promote ED's consent infrastructure as a regional benchmark

This is valid strength

SCROLL FOR ALL NINE ITEMS · NOTHING TRUNCATED

A quarter of a dollar per document pair

Cost per document-pair analysis, model by model. **Assumptions stated below, not measured.**

COST PER DOCUMENT-PAIR ANALYSIS

Assumptions, not measured

STAGE	MODEL	ASSUMED VOLUME	ASSUMED TOKENS	COST
Stage 3: Batched same-topic finder	Sonnet 5 $3 / $15 per MTok	8 calls 60 candidates ÷ 8/call	~3,000 in / 800 out per call	**$0.168**
Stage 5: Whole-doc coverage finder	Opus 4.8 $5 / $25 per MTok	1 call	~8,000 in / 1,500 out	**$0.078**
Total per document pair				**~$0.25**
Stage 1: Axis extraction *One-time per document, cached*	Haiku 4.5 $1 / $5 per MTok	1 call per document	~2,000 in / 200 out	**$0.003**/doc

Deterministic pipeline stages. Stages 2, 4 and 6 are deterministic code — retrieval, suppression, merge / citation-validation — with **no model cost**.

Corpus generation. The entire committed demo corpus — 9 recorded document pairs plus one-time extraction across 9 documents — costs **under $3 total**, which is why the demo strategy is build-and-persist, not live-on-stage.

Copilot Q&A. A turn (Sonnet, ~4,000 in / 500 out) runs about **$0.02** on the live backend already built — cheap enough for interactive use once the UI is wired to it. **Today's demo runs the copilot on pre-filled data**, so this cost is not incurred on stage.

MVP cost today. Effectively negligible — low single-digit dollars in model spend for the whole committed fixture set, plus whatever the FastAPI + Vite hosting costs during the hackathon window.

Corpus generation for the whole committed demo costs **under $3 total**

Next: the production estimate at ~200 users

Roughly $1,700 a month to run this for everyone

Production estimate scaled to the whole target user base on AWS hosting. **~200–240 policy staff across five to six departments.**

MODEL INFERENCE	VOLUME / MONTH	COST / MONTH
Document-pair analyses	~4,330 (200 users × 5/week × 4.33)	**~$1,080**
Copilot turns	~17,300 (200 users × 20/week × 4.33)	**~$345**
Embeddings (retrieval index)	Re-indexing new / changed documents	**~$50**
Model inference subtotal	5 new analyses + 20 copilot turns per user, per week	**~$1,500**

Claude Platform on AWS Recommended

Same-day parity with the first-party API and **the same pricing table above** — the lower-migration-risk match for what's already built.

Amazon Bedrock

Its own release cadence and a **separate pricing table**. Don't assume 1:1 parity — check Bedrock's own pricing before committing.

NON-MODEL AWS INFRASTRUCTURE	PURPOSE	COST / MONTH
ECS Fargate *or Lambda, given spiky usage*	FastAPI backend	**~$50–70**
S3 + CloudFront *frontend, plus corpus / clause index / findings JSON*	Vite / React static hosting	**~$15–20**
RDS (db.t3.micro) *or Aurora Serverless v2*	Real DB, if multi-user review state needs one	**~$50–100**
ALB + CloudWatch	Load balancing, logs / metrics	**~$30**
		~$150–

Claude Platform on AWS is the lower-migration-risk option if AWS hosting is a hard requirement

engine/config.py already targets first-party Claude behaviour

Where the engine stops working well

Stated plainly, because a tool a regulator can defend is one whose boundaries are known before deployment rather than discovered after it.

01 LLM timeout when a document is too large

The **context window is limited**. Stage 5's whole-document coverage pass must hold both documents at once, so a sufficiently large pair will time out rather than degrade gracefully.

Bounded by the same design that makes coverage findings possible — **the trade is deliberate**, and the failure is loud rather than silent.

02 Non-PDF chunking may be poor, noisy and inconsistent

Chunking performance on **XLSX and PPTX** is materially worse than on PDF. These formats carry structure that a linear segmenter does not recover.

Doc-class-aware segmentation means **the drafter chooses the strategy** — the tool never guesses, because a wrong guess yields unusable passages.

03 Dependency on parsable document formats

Scanned documents with poor PDF quality and **complex column layouts** can degrade parsing quality. Everything downstream inherits that degradation.

Azure Document Intelligence recovers reading order on multi-column source. **The verbatim substring check still holds** — a badly parsed passage cannot be quoted as if it were clean.

04 Coverage findings may be noisy across topically distant documents

`silent-on` and `goes-beyond` may be **noisy on documents that are related but topically differ** — the pass looks for absence, and absence is abundant between distant documents.

Stage 4 suppression removes ground Stage 3 already settled, and **the drafter accepts or rejects every finding** before it reaches a recommendation.

These are **production boundaries**, distinct from the prototype's own scope — see next slide

Known before deployment, not discovered after

What is demonstrated, and what is scripted

Three honest boundaries on what is being shown today. Each names the built capability behind it, so the distance from prototype to production is visible rather than implied.

BOUNDARY 01

Copilot and rulebook are currently running with pre-filled data

The slash-command workflow is **scripted in the UI** and grounded in verbatim fixture clauses. The live model backend behind it — real `call_chat`, streaming SSE, the `_validate_reply` citation guardrail, playbook-stage injection — **is built and tested**; only the command→stage dispatch is missing.

• Backend built | Orchestration pending

BOUNDARY 02

Currently supports PDF only — not Excel, not PPTX

The ingestion path is proven end-to-end on **PDF**. XLSX and PPTX are out of scope for this prototype, and their chunking quality is a known production limitation rather than a solved case.

Phase 2: M365 / SharePoint pulls PDF, PPTX, XLSX directly

BOUNDARY 03

Chunking strategy is selected, not routed

Three segmenter strategies are implemented — **structured-rules**, **unstructured**, **prose** — and exposed as a frontend selection. There is **no routing logic**: the drafter picks the strategy for the document class.

By design — a wrong automatic guess yields unusable passages

The through-line

Every boundary above is a **scope decision**, not a hidden defect. The guarantee that makes SELARAS defensible — **every claim traceable to a verbatim clause, enforced in code** — holds in the prototype exactly as it would in production, because it is the same validator either way.